

MAE014 - Tóp. Ciência Dados B [Turmas MAP e EMA]

Preenchimento de Região em Matriz de Caracteres

Nome completo	DRE
Emanuel Cardoso	122033151
Alanna Figueiredo Simões	120053919
Felipe Rodrigues Schoeffer	123311188
Artur de Melo Brito	117224234
Matheus de Sousa	123536584

Data: 11/09/2026

POST_MORTEM — Preenchimento de Região em Matriz de Caracteres

1. Objetivo

Implementar o algoritmo de preenchimento de região ("flood fill", usado em ferramentas de balde de tinta como o Paint e em navegação de robótica) de duas formas — recursiva e com uma pilha explícita de posições (linha, coluna) — reaproveitando a classe `Pilha` de `pilha.py`. A matriz é lida de um arquivo, exibida antes e depois do preenchimento, e o usuário pode acompanhar a evolução do algoritmo em blocos de P passos ($P=0$ executa tudo de uma vez). Também foi pedida uma interface gráfica (bitmap de pixels) com botões de Play (a X iterações por segundo) e Passo, além de um mecanismo de exibição único, independente do algoritmo utilizado.

2. Log de iteração e prompts

Prompt 1 — enunciado do exercício

"[...] Desenvolva dois programas, um que utiliza uma rotina recursiva, e outro que utiliza uma pilha de posições (linha, coluna), para preencher uma região de uma matriz de caracteres [...] A matriz de caracteres deve ser lida de um arquivo e ser apresentada na tela antes e depois do preenchimento. Permita que o usuário especifique uma quantidade P de passos a serem executados entre cada apresentação da matriz [...] Se P for zero, então a rotina deverá executar até o fim sem paradas intermediárias [...] Melhore esta entrada e saída com um BITMAP, ou seja, uma matriz de Pixels. Também aceite cores diversas [...]"
(texto integral do enunciado, colado pelo usuário, incluindo os três exemplos de matriz reproduzidos em matrizes/).

Prompt 2 — requisitos adicionais do grupo

"Repare que ele pediu que o algoritmo fosse implementado de duas formas diferentes [...] mas o mecanismo de plot do passo a passo deve ser único, independente do algoritmo utilizado. Se você conseguir, gostaria que você gerasse uma janela de interface que simulasse as iterações do algoritmo [...] um botão de play [...] X iterações por segundo (sendo X um input recebido pelo terminal em run time antes de renderizar a janela) [...]"
Além disso [...] um botão de play por iteração [...] Modularize bem o que é algoritmo, interface e renderização de janela [...]"

Esses dois prompts definiram, respectivamente, o comportamento funcional exigido pelo enunciado e a arquitetura modular pedida pelo grupo (separação entre algoritmo, reprodução em texto e reprodução gráfica).

3. Código inicial gerado pela IA (bruto, antes da revisão)

O primeiro rascunho, obtido a partir do Prompt 1, seguia o caminho mais direto para cada um dos dois algoritmos pedidos. Os fragmentos abaixo são o código bruto (sem qualquer edição manual), preservados aqui para transparência — a versão final entregue está em `preenchimento_pilha.py` e `preenchimento_recursoivo.py`, e é diferente da mostrada abaixo pelos motivos detalhados no `CODE_REVIEW.md`.

Preenchimento com pilha (rascunho bruto — duas pilhas paralelas):

```
def preencher(matriz, linha, coluna, novo="0", alvo="1"):
    n_linhas, n_colunas = len(matriz), len(matriz[0])
    capacidade = n_linhas * n_colunas * 4
    pilha_linhas = Pilha(capacidade, int)
    pilha_colunas = Pilha(capacidade, int)
    pilha_linhas.empilha(linha)
    pilha_colunas.empilha(coluna)

    while not pilha_linhas.pilha_esta_vazia():
        l = pilha_linhas.desempilha()
        c = pilha_colunas.desempilha()
        if l < 0 or l >= n_linhas or c < 0 or c >= n_colunas:
            continue
        if matriz[l][c] != alvo:
            continue
        matriz[l][c] = novo
        for dl, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            pilha_linhas.empilha(l + dl)
            pilha_colunas.empilha(c + dc)
```

Preenchimento recursivo (rascunho bruto — matriz como lista de strings, sem proteção de recursão):

```
def preencher(matriz, linha, coluna):
    n_linhas, n_colunas = len(matriz), len(matriz[0])
    if linha < 0 or linha >= n_linhas or coluna < 0 or coluna >= n_colunas:
        return
    if matriz[linha][coluna] != "1":
        return
    matriz[linha] = matriz[linha][:coluna] + "0" + matriz[linha][coluna + 1:]
    for dl, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        preencher(matriz, linha + dl, coluna + dc)
```

Nenhum dos dois rascunhos trazia mecanismo de reprodução por passos, interface gráfica ou testes — isso foi construído na revisão, já com a arquitetura modular pedida no Prompt 2.

4. Code Review crítico

Ver `CODE_REVIEW.md` para a análise completa. Resumo dos problemas encontrados nos dois rascunhos:

- 1 **Pilha (duas pilhas paralelas):** empilha os 4 vizinhos sem checar se

ainda são alvo, validando só ao desempilhar — gera muito mais operações de pilha do que o necessário e obriga a uma capacidade "estimada" por excesso ($\times 4$), que não é uma garantia real.

1 Recursivo (lista de strings): cada célula preenchida reconstrói a linha inteira via fatiamento (strings são imutáveis em Python) — custo extra por escrita.

1 Recursivo (sem proteção de recursão): o Python tem um limite padrão de 1000 chamadas recursivas; regiões grandes e abertas (poucas bifurcações) estouram esse limite bem antes de faltar memória de verdade — um problema real, não hipotético (medido nos testes de estresse).

5. Justificativa da refatoração

- **Uma única Pilha de inteiros codificados** ($\text{linha} * \text{n_colunas} + \text{coluna}$) no lugar de duas pilhas paralelas, e uma célula só é empilhada quando descoberta como alvo (marcando-a novo no mesmo instante). Isso garante no máximo um `empilha()` por célula, permitindo a capacidade exata $\text{n_linhas} * \text{n_colunas}$ — sem estimativas.
- **Matriz como lista de listas de caracteres**, não lista de strings, para que a escrita de uma célula seja $O(1)$ em vez de reconstruir a linha inteira.
- **Execução da recursão em uma thread dedicada**, com pilha nativa maior (64 MiB) e limite de recursão do interpretador ajustado ao tamanho da matriz. Sem isso, a versão recursiva simplesmente não teria como resolver regiões grandes — não é uma otimização de conforto, é o que torna a rotina recursiva utilizável para o caso geral pedido no enunciado.
- **Mecanismo de reprodução único** (`reprodutor.py` para texto, `interface_grafica.py` para a janela gráfica): ambos os algoritmos passaram a devolver apenas a lista ordenada de posições preenchidas (a "matriz" propriamente dita é alterada em memória, e a lista de eventos é o que os reprodutores usam para tocar a evolução do algoritmo passo a passo, com pausas a cada P eventos ou continuamente se $P = 0$). Isso satisfaz diretamente o pedido do Prompt 2 de que a exibição não dependa de qual algoritmo gerou o preenchimento.

6. Evidência de testes

Resumo (números completos em `RELATORIO_TESTES.md`):

- A versão final da pilha faz **4× menos operações de empilhar** e roda **cerca de 3× mais rápido** que a versão ingênua de duas pilhas, no mesmo

cenário de teste — e chega ao mesmo resultado.

- A versão recursiva final resolve corretamente uma sala de 250×250 (62.500 células), enquanto a versão sem proteção de recursão **estoura** (`RecursionError`) já em uma sala de 60×60 (3.600 células).
- Os dois algoritmos (recursivo e com pilha) produzem exatamente o mesmo resultado final em todos os exemplos testados, incluindo os três exemplos do próprio enunciado.

7. Conclusão

A IA acelerou a primeira versão funcional dos dois algoritmos, mas o código bruto tinha exatamente os problemas que o enunciado pede para identificar: ineficiência de operações de pilha, complexidade de escrita em string e um limite de recursão que a IA não tratou (o rascunho bruto simplesmente ignora essa limitação até que um input real o quebre). A etapa de revisão foi necessária tanto para a corretude (capacidade exata da pilha, ausência de vazamento em bordas abertas) quanto para o desempenho e para atender ao requisito de modularização entre algoritmo, reprodução em texto e interface gráfica.